

NAANAA

Healthy Meal Delivery System

System Analysis and Design Document

Course:	System Analysis and Design
Project Model:	Waterfall SDLC
Prepared by:	Peaky Blinders Team
Date:	May 2026
Supervisor:	Dr. Abeer

Team Name: Peaky Blinders Code
Team Leader: Mohamed Essam Mohamed Ibrahim

Team Members:

Mohamed Essam Mohamed Ibrahim	Mohamed Ahmed Almaz	Ziad Ateyya Ateyya
Malek Osama Mohamed Mahmoud	Mohamed Saad Abady Hamid	Mark Mina Sidhom Farid
Aya Ahmed Mohamed Hussein	Ahmed Samy Mohamed Abu Bakr	Mohamed Alaa El-Din Hassan
Youssef Mostafa Mohamed Abd El-Wahab	Mazin Hassan Ali Hassan Ahmed	Mahmoud Mohamed Mohamed Ahmed

Phase 1: Problem Identification

Understanding the Challenge

1.1 Problem Definition

In the rapidly accelerating environment of Alexandria, students and professionals face significant challenges in maintaining a balanced, healthy diet. While the local market is saturated with conventional fast-food delivery applications, there is a distinct lack of platforms dedicated to nutritious, macro-calculated, and diet-specific meals. Users who wish to pursue fitness goals, manage dietary restrictions, or simply eat cleaner are forced to navigate fragmented menus, call restaurants directly, or settle for unhealthy alternatives.

The **Naanaa** project addresses this gap by introducing a streamlined Business-to-Consumer (B2C) online food ordering system strictly focused on healthy eating. By providing a centralized platform, Naanaa aims to eliminate the friction associated with finding and ordering healthy food, offering complete transparency regarding nutritional values, ingredients, and preparation methods.

1.2 Impact on Stakeholders

The absence of a specialized healthy food platform creates a cascading negative impact across multiple stakeholder groups:

- **Customers (Students & Professionals):** Compromised health, increased fatigue, and the psychological burden of failing to meet personal fitness or medical dietary goals due to inconvenience and lack of accessible healthy options.
- **Health-Conscious Restaurants:** Struggle with visibility and customer acquisition in a marketplace dominated by heavily funded fast-food chains. They lack a dedicated channel to reach their target demographic.
- **Healthcare Providers & Nutritionists:** Face difficulties in ensuring patient compliance with prescribed diets, as patients lack practical, accessible tools to order the recommended meals.

1.3 Background and Justification

The necessity for the Naanaa system is justified by the rising global and local awareness of nutritional health. With the increasing prevalence of lifestyle-related conditions, there is a measurable shift in consumer behavior toward cleaner eating. Developing Naanaa utilizing a robust technology stack—specifically a **React** frontend for a highly responsive single-page application (SPA) experience and a secure, scalable **Django** backend—ensures that the platform can handle complex data relationships (like macro tracking and user profiles) while delivering a seamless user experience. The structured **Waterfall SDLC** model has been adopted to ensure rigorous requirement gathering, logical progression, and meticulous documentation from zero to deployment.

Phase 2: Information Gathering

Interviews, Analysis & Requirements

2.1 Stakeholder Interviews

To establish a solid foundation for the system's requirements, the development team conducted comprehensive interviews with six key stakeholders representing different facets of the ecosystem. The interviewees included three targeted end-users (university students and working professionals), one restaurant manager specializing in healthy meal prep, one delivery logistics coordinator, and one dietary consultant.

Interview Methodology: The Pyramid Structure

The interviews were designed utilizing a **Pyramid Questionnaire Structure**. This approach begins with highly specific, closed-ended questions to establish a baseline of current behaviors, and gradually expands into broad, open-ended questions to uncover deep insights and undiscovered needs.

2.2 Interview Analysis and Insights

The data extracted from the interviews yielded several critical insights that directly shaped the system's architectural and functional planning:

- **Transparency is Paramount:** Users expressed profound frustration with the ambiguity of current menus. A core expectation for Naanaa is the explicit listing of calories, proteins, carbohydrates, and fats for every menu item.
- **Streamlined Kitchen Operations:** The restaurant manager highlighted that handling customized healthy orders requires precise instructions. The system must provide a clear, real-time dashboard for kitchen staff to view order specifics and dietary notes.
- **Delivery Logistics:** The delivery coordinator emphasized the need for live location updates and direct communication channels with the customer to ensure meals are delivered fresh and within precise time windows.
- **Backend Reliability:** Due to the anticipated volume of customized queries (filtering by macros, allergens, etc.), the backend architecture must be logically structured and highly efficient. Django ensures robust relational database management to handle these complex queries effortlessly.

2.3 System Requirements Analysis

Functional Requirements	Non-Functional Requirements
• User Authentication: Secure registration and login for Customers, Kitchen Staff, Delivery Personnel, and Administrators.	• Performance: React frontend renders menu changes in under 2 seconds. Django backend API responds to queries within 200ms.

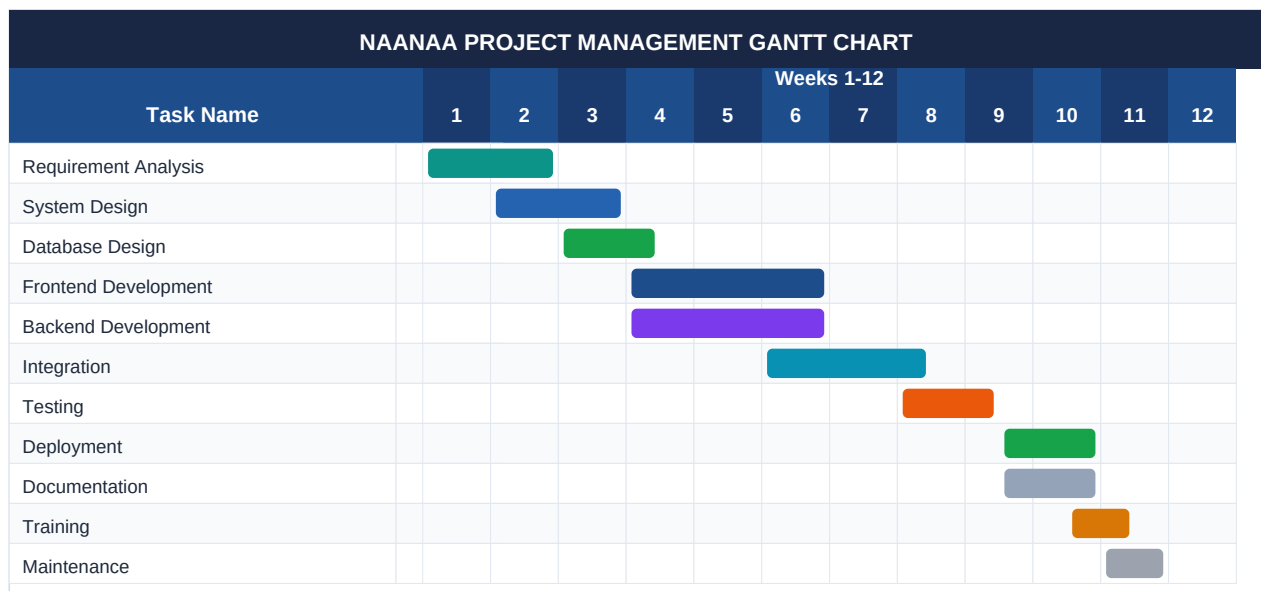
• Menu Browsing & Filtering: Dynamic filtering by nutritional content, ingredients, allergens, and item availability.	• Security: All user passwords hashed using bcrypt. Payment data encrypted in transit and at rest using AES-256.
• Order Management: Full cart functionality, secure payment gateway integration, order placement, tracking, and cancellation.	• Scalability: System architecture supports horizontal scaling to accommodate peak ordering hours and growing user base.
• Kitchen Dashboard: Real-time interface for kitchen staff to receive, acknowledge, prepare, and dispatch orders.	• Availability: Target 99.5% uptime. Graceful degradation during partial failures ensures users can still browse the menu.

Phase 3: Project Scheduling

Gantt Chart & PERT Network Analysis

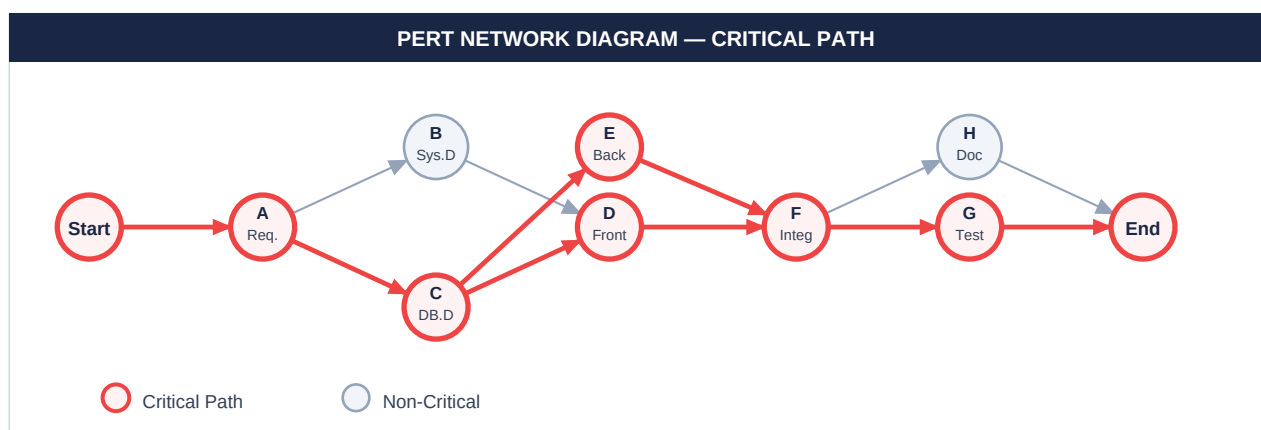
3.1 Project Timeline — Gantt Chart

The Naanaa project strictly follows the Waterfall SDLC model, segmented over a **12-week development cycle**. The Gantt chart below details durations for all phases: Requirement Analysis, System Design, Database Design, Frontend (React), Backend (Django), Integration, Testing, Deployment, Documentation, Training, and Maintenance. Parallel development of Frontend and Backend phases (Weeks 4–6) represents the most critical scheduling block.



3.2 PERT Network Diagram — Critical Path Analysis

The PERT network diagram maps the critical path (highlighted in red) and task dependencies. Any delay in nodes on the critical path directly impacts the final delivery date. Non-critical paths offer float time that can absorb minor delays without affecting the overall schedule.



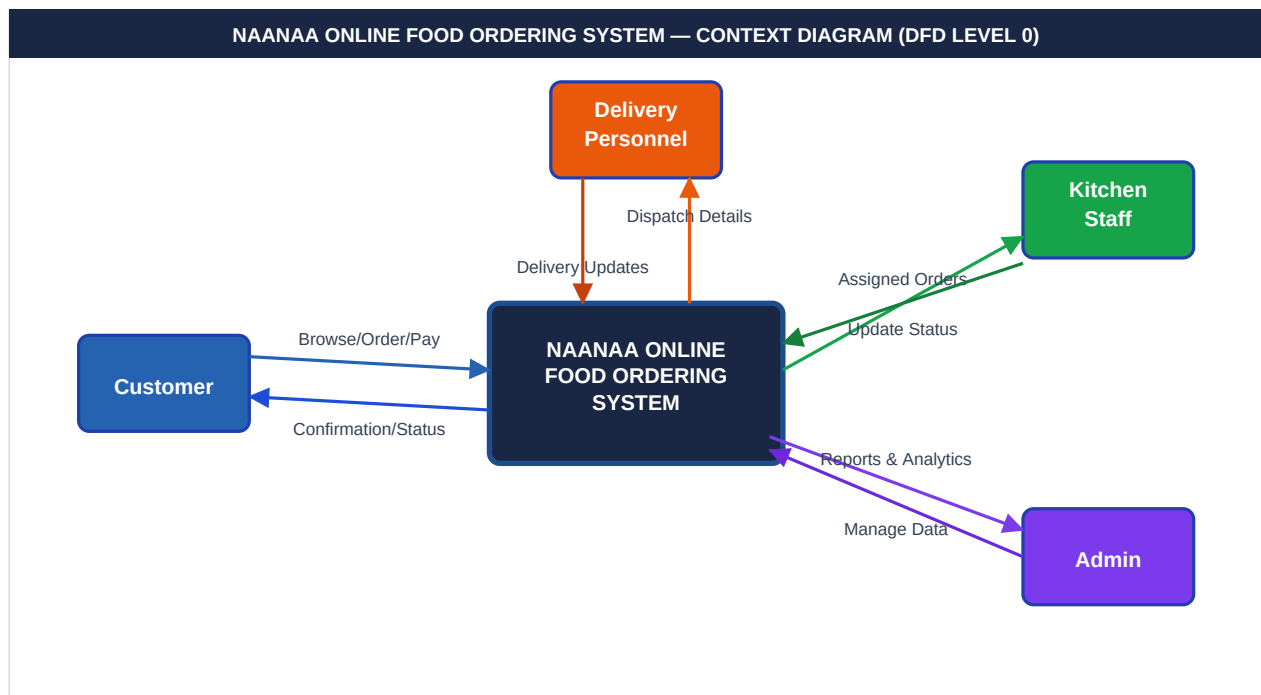
Explanation: As illustrated above, the parallel development of Frontend and Backend phases (spanning Weeks 4 to 6) is the most critical block. Any delay in the preceding Database Design phase directly impacts this block, subsequently delaying Integration. The PERT network diagram explicitly defines these strict sequential dependencies and identifies the critical path running: Start → A → C → D → E → F → G → End.

Phase 4: System Modeling — Data Flow

DFD Context, Level 0 & Level 1

4.1 Context Flow Diagram (DFD Level 0 — Context)

The Context Diagram establishes the highest-level view of the Naanaa system, outlining the boundary of the platform and how data flows to and from the primary external entities: **Customer**, **Admin**, **Kitchen Staff**, and **Delivery Personnel**.

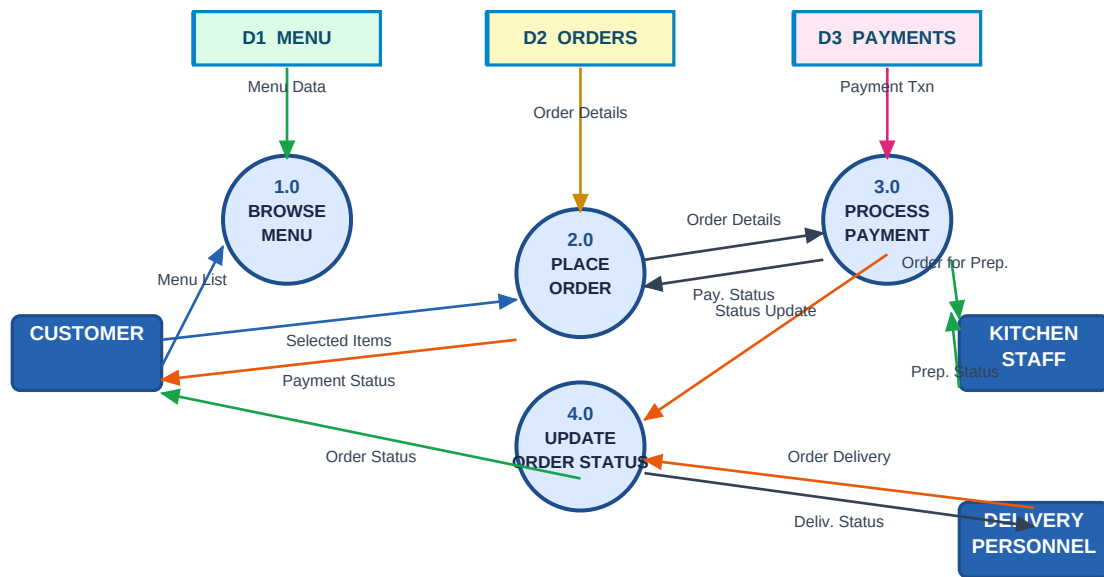


Explanation: The central node represents the Naanaa application itself. Customers send inputs such as browsing requests, order placements, and payments, receiving confirmations and status updates in return. Meanwhile, the system automatically routes assigned orders to Kitchen Staff and dispatch details to Delivery Personnel, whilst sending administrative reports and analytics to the Admin entity.

4.2 Level 0 Data Flow Diagram

Breaking down the central system, the Level 0 DFD illustrates the four major internal processes: **1.0 Browse Menu**, **2.0 Place Order**, **3.0 Process Payment**, and **4.0 Update Order Status**.

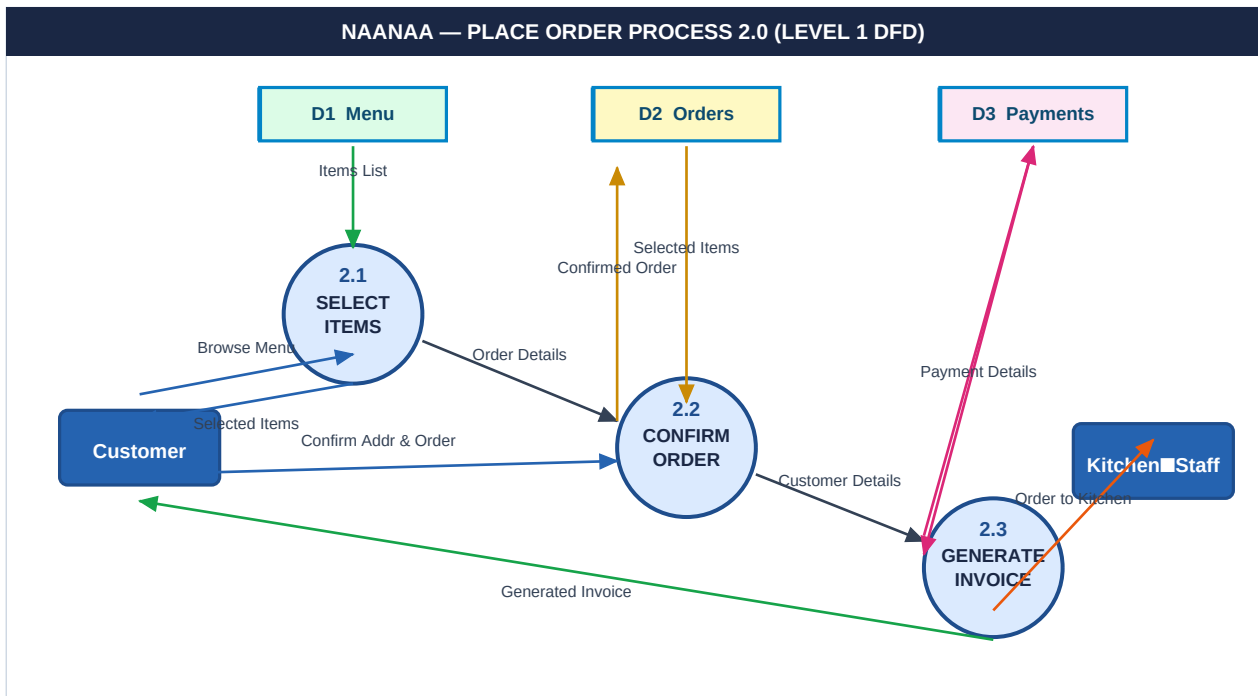
NAANAA LEVEL 0 DATA FLOW DIAGRAM



Explanation: This diagram reveals the interaction with the core datastores (D1: Menu, D2: Orders, D3: Payments). Process 2.0 (Place Order) accepts the Customer's selected items and pushes the pending data into the Orders datastore. Process 4.0 acts as a central hub for asynchronous status updates from Kitchen and Delivery, routing the live tracking data back to the Customer.

4.3 Level 1 Data Flow Diagram — Place Order Detail

Zooming into Process 2.0, the Level 1 DFD delineates the sub-processes involved in converting a cart into a confirmed order ticket: **2.1 Select Items**, **2.2 Confirm Order**, and **2.3 Generate Invoice**.



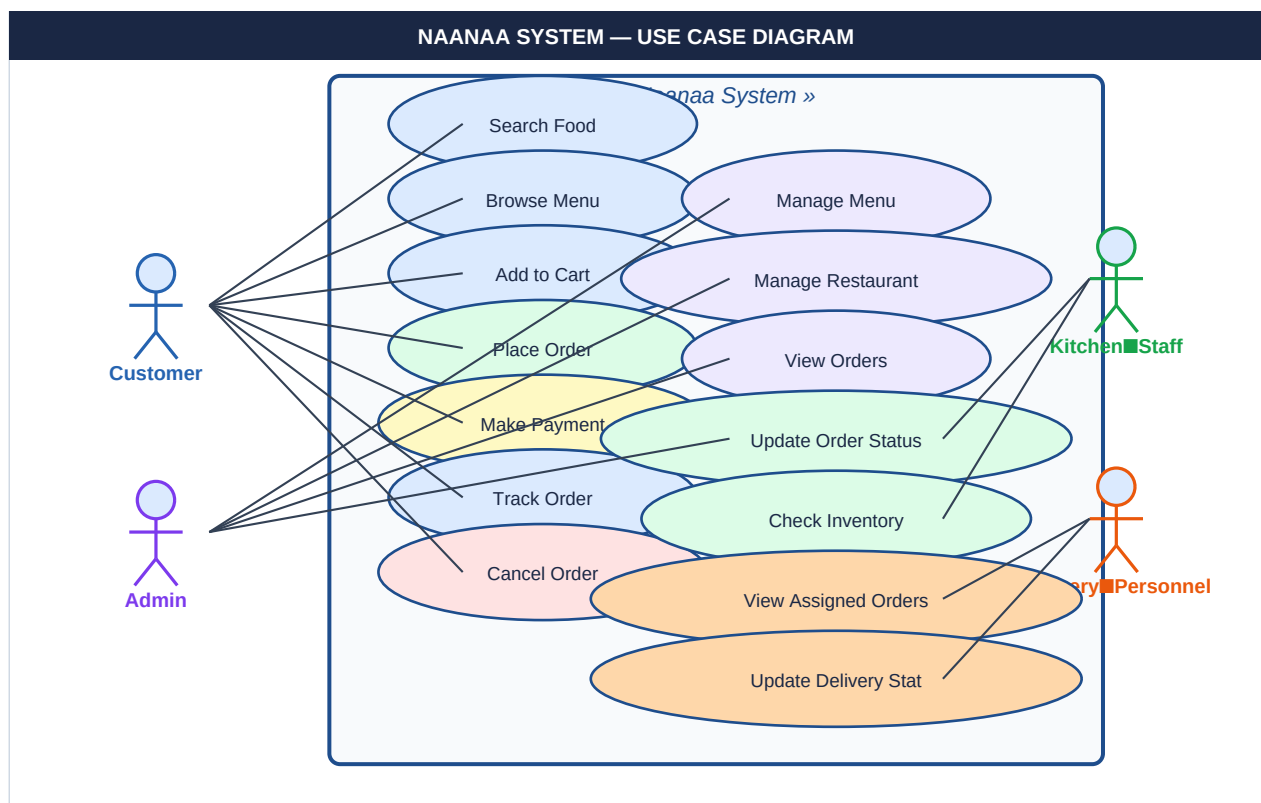
Explanation: The procedure is split into Select Items (2.1), Confirm Order (2.2), and Generate Invoice (2.3). The final invoice generation is contingent upon successful payment confirmation, after which the generated ticket is immediately dispatched to the Kitchen Staff, ensuring seamless operational flow without skipping validation steps.

Phase 5: UML Diagrams

Use Case & Class Diagram

5.1 Use Case Diagram

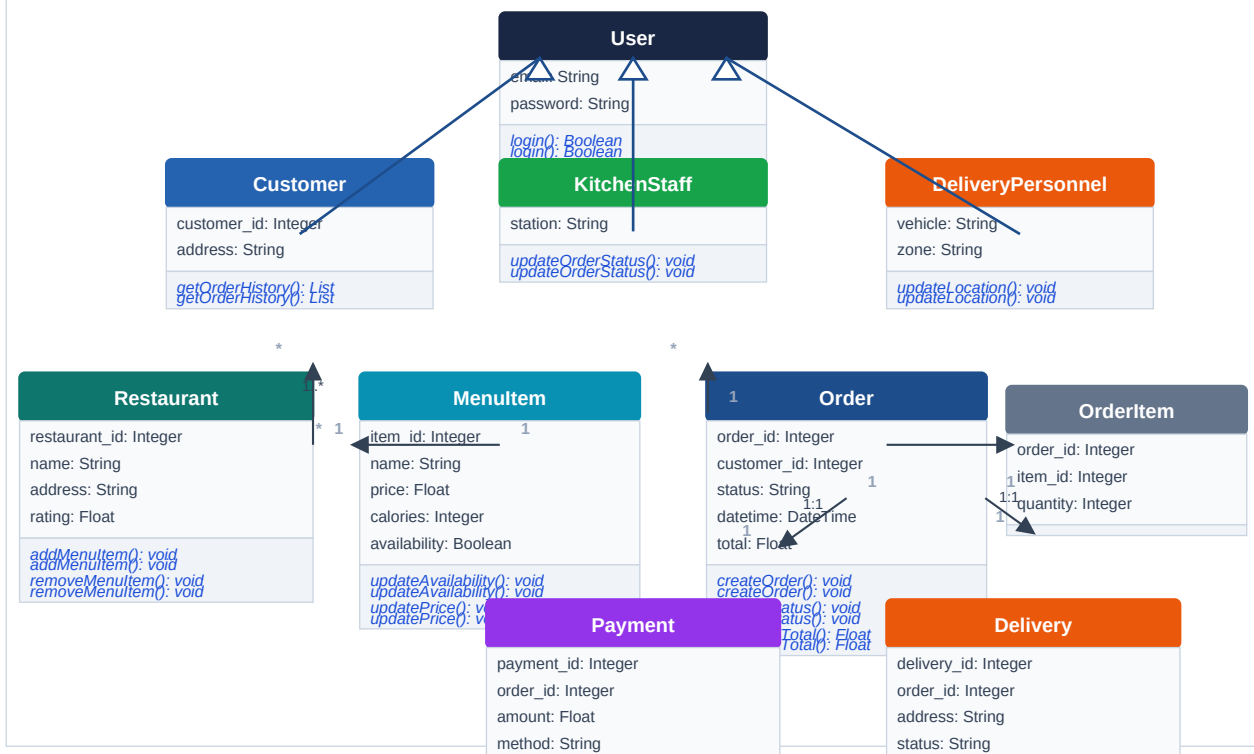
The Use Case diagram visually maps the functional boundaries of the system by connecting specific actors to the actions they are authorized to perform within the platform. Four primary actors interact with the system: Customer, Admin, Kitchen Staff, and Delivery Personnel.



Explanation: The system successfully isolates administrative functionality (managing the menu, viewing all orders) from standard operations. The Customer has a broad array of interactions focused on the ordering funnel, while Kitchen Staff and Delivery Personnel are restricted to targeted use cases like updating statuses and checking inventory.

5.2 Class Diagram

The Class Diagram serves as the structural blueprint for the Django backend object-relational mapping (ORM) layer, detailing entities, attributes, methods, and their relationships.



Explanation: The diagram showcases an effective inheritance hierarchy where the User class is abstracted into Customer, KitchenStaff, and DeliveryPersonnel. Critical relationships include the strict 1-to-many relationship between a Customer and an Order, and the 1-to-1 strict mapping between Order, Payment, and Delivery. The MenuItem class encapsulates vital nutritional data necessary for the core filtering capabilities of the platform.

Phase 6: Data Dictionary

Dr. Abeer's Requirement — May 2026

As per the instructor's updated requirements (Lecture — May 2026), this phase presents the formal **Data Dictionary** for the Naanaa system. The data dictionary documents the precise definition of all data components used in the system. It is organized into four complementary sections: **Data Flows**, **Data Elements**, **Data Structures**, and **Data Stores**.

Data Flows	Data Elements	Data Structures	Data Stores
Documents how data moves between processes, actors, and stores within the system.	Defines individual data fields: their type, format, constraints, and valid values.	Shows how elements combine into composite objects passed through the system.	Describes persistent storage locations — their purpose, contents, and access rules.

6.1 Data Dictionary — Data Flows

Data flows represent the movement of information between processes, external entities, and data stores. Each flow is characterized by its source, destination, the data elements it carries, its direction type, and the frequency at which it occurs in the system.

Flow Name	From	To	Data Elements	Type	Frequency
Browse Menu Request	Customer	System	user_id, filter_params, category	Input	On-demand
Menu List	D1: Menu Store	Customer	item_id, name, price, calories, macros, availability	Output	On-demand
Place Order	Customer	2.0 Place Order	cart_items[], quantity[], special_notes	Input	Per transaction
Order Confirmation	System	Customer	order_id, status, estimated_time, total_price	Output	Per transaction
Assigned Orders	System	Kitchen Staff	order_id, items[], dietary_notes, timestamp	Output	Real-time
Update Status	Kitchen Staff	System	order_id, status_code, prep_time_remaining	Input	Real-time
Delivery Details	System	Delivery Personnel	order_id, customer_address, route_info	Output	Per dispatch
Delivery Updates	Delivery Personnel	System	delivery_id, gps_coords, eta	Input	Real-time
Payment Data	Customer	3.0 Process Payment	card_info (encrypted), amount, currency	Input	Per transaction
Reports & Analytics	System	Admin	sales_data, order_counts, peak_hours, revenue	Output	Scheduled/On-demand

6.2 Data Dictionary — Data Elements

Data elements are the atomic, indivisible units of data stored and processed by the Naanaa system. Each element is defined by its name, description, data type, format/length constraints, validation rules, and a representative example value.

Element Name	Description	Type	Length/Format	Constraints	Example
user_id	Unique user identifier	Integer	Auto-increment	NOT NULL, PK	1042
email	User email address	String	max 255 chars	UNIQUE, NOT NULL	ali@naanaa.eg
password	Hashed password (bcrypt)	String	60 chars (hash)	NOT NULL, Hashed	\$2b\$12\$...
item_id	Menu item unique ID	Integer	Auto-increment	NOT NULL, PK	307
item_name	Display name of food item	String	max 100 chars	NOT NULL	Grilled Salmon Bowl
calories	Caloric value (kcal)	Integer	0–9999	NOT NULL, >= 0	520
price	Item price in EGP	Float	2 decimal places	NOT NULL, > 0	145.50
macros_protein	Protein content (grams)	Float	0.0–999.9	NOT NULL, >= 0	38.5
macros_carbs	Carbohydrate content (grams)	Float	0.0–999.9	NOT NULL, >= 0	42.0
macros_fat	Fat content (grams)	Float	0.0–999.9	NOT NULL, >= 0	18.2
availability	Item availability flag	Boolean	True/False	NOT NULL, Default True	True
order_id	Unique order identifier	Integer	Auto-increment	NOT NULL, PK	20041
order_status	Current order lifecycle state	Enum	max 20 chars	pending confirmed preparing dispatched delivered cancelled	preparing
order_datetime	Timestamp of order creation	DateTime	ISO 8601	NOT NULL, Auto-set	2026-05-01T14:32:00Z
total_price	Computed cart total (EGP)	Float	2 decimal places	NOT NULL, computed	290.00
delivery_addr	Customer delivery address	String	max 300 chars	NOT NULL	42 Corniche, Alex
payment_method	Payment channel used	Enum	max 20 chars	cash card wallet	card
payment_status	Result of payment processing	Enum	max 20 chars	pending paid failed refunded	paid
dietary_notes	Customer dietary restrictions	String	max 500 chars	Optional	No gluten, extra protein
allergen_flags	Allergen tags on menu item	JSON	Array of strings	Optional	["nuts","dairy"]

6.3 Data Dictionary — Data Structures

Data structures define how individual elements are grouped and organized into composite objects that are passed through the system's processes. They represent the logical grouping of related data elements that together form meaningful information units.

Structure Name	Component Elements	Relationship	Notes
Customer Profile	user_id + email + password + customer_id + name + phone + address[] + dietary_preferences + order_history[]	Extends User base entity	Stored across Users (D4) and Orders (D2) stores. Preferences used for meal filter recommendations.
Menu Item Record	item_id + restaurant_id + name + description + price + calories + {protein, carbs, fat, fiber} + allergen_flags[] + availability + image_url	Belongs to Restaurant; linked to OrderItem	Nutritional sub-structure (macros) is a nested object. Allergens stored as JSON array for flexible filtering.
Order Object	order_id + customer_id + restaurant_id + order_items[{item_id, qty, notes}] + status + timestamps{created, confirmed, dispatched, delivered} + total + delivery_address + special_instructions	References Customer, Restaurant, MenuItem; has one Payment, one Delivery	Status field follows a strict FSM: pending → confirmed → preparing → dispatched → delivered.
Payment Transaction	payment_id + order_id + amount + currency + method + gateway_transaction_ref + status + created_at + updated_at	One-to-one with Order	Gateway reference used for refund and dispute tracking. Payment data is encrypted at rest.
Delivery Record	delivery_id + order_id + personnel_id + pickup_address + drop_address + gps_trail[] + status + assigned_at + delivered_at + customer_rating	One-to-one with Order; belongs to DeliveryPersonnel	GPS trail stored as array of timestamped coordinates for live tracking and post-delivery analysis.

6.4 Data Dictionary — Data Stores

Data stores are persistent repositories where data is held between processes. In the Naanaa system, six primary data stores are identified, each corresponding to a distinct domain of the application. The store IDs (D1–D6) align with those used in the DFD diagrams in Phase 4.

Store ID	Store Name	Description	Key Elements	Access	Volume Est.
D1	Menu Store	Persistent storage of all menu items, nutritional data, pricing and availability managed per restaurant	item_id, name, price, calories, macros, allergens, restaurant_id, availability	Read: All Users Write: Admin, Kitchen	~500 items
D2	Orders Store	Central record of all customer orders from placement through to delivery completion with full audit trail	order_id, customer_id, items[], status, timestamps, total, delivery_address	Read: Customer(own), Admin, Kitchen, Delivery Write: System	~10K/month

Store ID	Store Name	Description	Key Elements	Access	Volume Est.
D3	Payments Store	Encrypted financial transaction records linked to orders, supporting multiple payment methods with refund tracking	payment_id, order_id, amount, method, status, gateway_ref, timestamp	Read: Customer(own), Admin Write: Payment Gateway	~10K/month
D4	Users Store	Accounts for all system actors: Customers, Kitchen Staff, Delivery Personnel, and Administrators with role-based data	user_id, email, password_hash, role, profile_data, address, active_flag	Read: Self, Admin Write: User, Admin	~5K users
D5	Delivery Store	Real-time and historical tracking of delivery assignments, routes, and completion status for all dispatched orders	delivery_id, order_id, personnel_id, gps_trail, status, timestamps, rating	Read: Customer(own), Admin, Personnel Write: System, Personnel	~10K/month
D6	Restaurants Store	Profiles of partner restaurants including location, operating hours, cuisine type, and performance ratings	restaurant_id, name, address, hours, cuisine_type, rating, active	Read: All Users Write: Admin	~50 partners

Phase 7: Bonus Requirements & Conclusion

Sequence Diagram, Feasibility & Summary

7.1 Sequence Diagram Outline — Checkout Process

To trace the real-time execution flow during the critical **"Checkout"** process:

1	The Customer UI sends a POST request containing cart contents to the Order Controller.
2	The Order Controller validates the request against the Database, checking item availability and stock.
3	The Controller interfaces with the Payment API, submitting encrypted payment details.
4	Upon a 200 OK response from the Payment API, the Controller commits the final Order object to the database.
5	The system broadcasts a WebSocket event to the Kitchen Dashboard to begin preparation immediately.

7.2 Feasibility Study

Technical Feasibility	Economic Feasibility	Operational Feasibility
The selected technology stack (React frontend and Django backend) is highly robust and battle-tested. Containerized environments (Docker) or cloud-based IDEs completely negate local hardware constraints, enabling consistent development and deployment.	Serving the Alexandria market taps into a growing demographic of health-conscious consumers, minimizing initial marketing friction. Open-source stack eliminates licensing costs, keeping initial investment low.	The dedicated, straightforward dashboard interfaces for kitchen and delivery staff mean low friction during operational deployment, requiring almost zero training time. The system's intuitive UX reduces onboarding overhead significantly.

Conclusion

The **Naanaa Online Food Ordering System** represents a highly structured solution tailored to address a definitive real-world gap in Alexandria's food delivery market. By rigorously applying the Waterfall SDLC—moving progressively from problem definition through information gathering, scheduling, DFD modeling, UML design, and formal data dictionary documentation—every functional and non-functional requirement has been accurately captured and defined.

The integration of **React** for dynamic frontend rendering and **Django** for a secure, relational backend ensures that Naanaa will effectively serve its health-conscious users and operational stakeholders alike.

The newly completed Data Dictionary in Phase 6 provides the definitive reference for all data flows, elements, structures, and stores — establishing a solid foundation for database design and implementation in the next phases of development.

Prepared by the Peaky Blinders Team — Faculty of Business, Alexandria University — May 2026